

1. Overview & Learning Objectives

You have successfully mastered the basics of D3.js, including selections and basic charts. However, the true power of D3 lies in its ability to handle geospatial data, complex interactions, physics simulations, and time-based animations.

In this assignment, you are required to move beyond "static charts" and build **two fully interactive, production-grade dashboards** using real-world data. This assignment is designed to be challenging; it will require you to study the D3 documentation for Geographies, Forces, Transitions, Interactivity (Drag/Zoom), and Hierarchy.

2. General Constraints & Submission Rules

- **Pure D3 Only:** You may **not** use wrapper libraries (e.g., NVD3, Plotly, Chart.js) or framework-specific bindings (e.g., React-D3). All DOM manipulation must be done using standard D3 patterns (`enter/update/exit`).
 - **Code Structure:** Do not write all code in one file. Modularize your work (e.g., `map.js`, `charts.js`, `main.js`, `utils.js`).
 - **Responsive:** Your visualizations must handle window resizing gracefully (use `viewBox`).
 - **Data Loading:** Data must be loaded asynchronously. Do not hardcode data into your JavaScript files.
-

Task 1: The Spatial & Network Analysis

Scenario: You have been contracted by the **International Energy Agency (IEA)**. They possess a massive dataset of power plants globally. They need a dashboard to visualize the distribution of energy capacity and investigate the mix of fuel types (Coal vs. Renewable) across different regions.

The Data:

- **Source:** World Resources Institute (WRI) - Global Power Plant Database.
- **Download:** [global_power_plant_database \(https://github.com/wri/global-power-plant-database/tree/master\)](https://github.com/wri/global-power-plant-database/tree/master)
- **Required Columns:** Latitude, Longitude, Primary Fuel, Capacity (MW), Country, Commissioning Year.

Requirement 1.1: The Zoomable Aggregate Map

- **API Focus:** `d3-geo`, `d3-zoom`, `d3-fetch`
- Render a World Map using a projection of your choice.
- **Challenge:** The dataset contains ~35,000 points. Rendering them all at once will crash the browser.
- **Logic:** Implement **Semantic Zooming**:
 - *Zoom Out:* Show summarized circles per country (Size = Total Capacity of that country).
 - *Zoom In:* When the user zooms past a certain threshold, the country circles must explode into individual circles representing specific power plants at their exact coordinates.

Requirement 1.2: Force-Directed Fuel Clustering

- **API Focus:** `d3-force`, `d3-scale`
- Create a floating bubble chart (Force Simulation) alongside the map.
- **Logic:**
 - Each node is a "Fuel Type" (e.g., Solar, Coal, Nuclear).
 - Use `d3.forceCollide` to prevent overlap.
 - **Interaction:** Clicking a "Fuel" node acts as a global filter. If "Solar" is clicked, the Map (Req 1.1) must update to show *only* Solar plants.

Requirement 1.3: The "Brushable" Timeline

- **API Focus:** `d3-brush`, `d3-shape`, `d3-axis`
- Create a Stacked Area Chart at the bottom showing capacity added per year.
- **Logic:** Implement a **Brush** selection.
 - The user drags a box over a time range (e.g., 1990–2010).
 - **Linked View:** As the user brushes, both the Map and the Force Layout must instantly update to display only the power plants built within that time window.

Task 2: The Temporal & Hierarchical Simulation

Scenario: The **United Nations Development Programme (UNDP)** wants to visualize the correlation between wealth (GDP) and health (Life Expectancy) over the last two centuries. Unlike a static report, they require an **animated** simulation that allows users to "play" history or drill down into specific regions.

The Data:

- **Source:** Gapminder (World Development Indicators).
- **Download:** [gapminder_combined](https://www.gapminder.org/data/) (<https://www.gapminder.org/data/>)
- **Structure:** An array of objects containing yearly stats for countries (GDP, LifeExp, Pop, Continent).

Requirement 2.1: The Motion Chart (Scatter Plot)

- **API Focus:** `d3-timer`, `d3-transition`, `d3-interpolate`
- Create a Scatter Plot:
 - **X-Axis:** GDP per Capita (Must use `d3.scaleLog`).
 - **Y-Axis:** Life Expectancy (Linear Scale).
 - **Radius:** Population (Sqrt Scale).
- **Logic:** Implement a **Play/Pause** button using `d3.interval`.
 - When played, the years must advance automatically (e.g., 1900 -> 1901 -> 1902).
 - The bubbles must smoothly transition to their new positions for the next year.

Requirement 2.2: The Synchronized Choropleth Map

- **API Focus:** `d3-geo`, `d3-scale-chromatic`
- Render a World Map using `d3.geoPath`.
- **Logic:** The map colors must represent **Life Expectancy**.

- **Synchronization:** This map must be strictly coupled to the Motion Chart (Req 2.1). If the chart is showing data for the year 1955, the map countries must be colored according to 1955 data. As the animation plays, the map colors must shift in real-time.

Requirement 2.3: Interactive State Controls

- **API Focus:** `d3-drag`, `d3-hierarchy`
- **The Slider:** Create a slider using `d3.drag`. Dragging the handle should stop the animation and "scrub" the visualization to the specific year.
- **The Drill-Down:** Create a generic hierarchy chart (Donut or Sunburst) showing *World -> Continent -> Country*.
 - Hovering a "Continent" slice must lower the opacity of all bubbles on the Scatter Plot that do not belong to that continent.

3. Grading Rubric (Total: 200 Points)

Component	Criteria	Points
Data Architecture	Efficient loading, parsing, and restructuring (grouping/nesting) of CSV and JSON data.	15
Advanced Maps	Correct use of Projections, GeoPaths, and Zoom/Pan behaviors (Task 1 & 2).	20
Interactivity	Working Brushing (Task 1) and Dragging/Slider (Task 2).	20
Animation Logic	Smooth transitions without DOM thrashing; Correct use of Timers and Interpolation.	20
Visual Encoding	Correct choice of Scales and color schemes.	15
UI/UX & Polish	Tooltips, Legends, Labels, and overall aesthetic presentation.	10
Assignment Report	Include snaps of each separable step during assignment for each task & explain How & Why.	50
Viva	You must be able to defend what you performed with the knowledge you have of the tasks you did and answering why.	50

4. Hints & Self-Study Guide

- **Performance:** For Task 1, if the map lags, look into `canvas` rendering or optimize your `selectAll` calls.
- **Updates:** For Task 2, review the "General Update Pattern" (Join -> Enter -> Update -> Exit). If you destroy and recreate the chart every frame, the animation will flicker.
- **Documentation:** You will specifically need to read the API references for `d3-geo`, `d3-zoom`, `d3-drag`, and `d3-brush`. These were not covered deeply in class—this is the research portion of the assignment.
- **Report:** Be vigilant about report preperation and submission, your report and viva will dictate your marks.